AAE 5206 Group Project
13 Mar 2026

Note:
This group project fundamentally aims to enable all students to have solid experience on tackling more complex and real-world problems using deep learning techniques and improve collaboration and presentation skills. In addition, innovation and critical thinking are strongly encouraged to explore open-ended questions.
Each group should have 4 -5 members. Each group only needs to submit, but it must include all team members' information (names and student IDs) in the code, report, and presentation.

Project Requirements and Marking Criteria

1) Project Overview

Implement an airplane image classification or object detection task using VGG16 or ResNet50 pretrained models in PyTorch. You need to submit the code, group report and take a presentation for the group project.

Task1:
Use PyTorch's pre-trained ResNet50 model to complete the airplane image classification task.
The specific weight of the model is ResNet50_Weights.IMAGENET1K_V2.
You need to read the papers of ResNet [1] and Faster R-CNN [4]. And introduce them in the report and presentation.

Task2:
Use PyTorch's pre-trained VGG16 model to complete the airplane image classification task.
The specific weight of the model is VGG16_Weights.IMAGENET1K_V1
You need to read the papers of VGG [2] and SSD [3]. And introduce them in the report and presentation.

Task3:
Use PyTorch's pre-trained Faster R-CNN_ResNet50 model to complete the airplane image object detection task.
The specific weight of the model is FasterRCNN_ResNet50_FPN_V2_Weights.COCO_V1
You need to read the papers of ResNet [1] and Faster R-CNN [4]. And introduce them in the report and presentation.

Task4:
Use PyTorch's pre-trained SSD300_VGG16 model to complete the airplane image object detection task.
The specific weight of the model is SSD300_VGG16_Weights.COCO_V1
You need to read the papers of VGG [2] and SSD [3]. And introduce them in the report and presentation.
The weights and example code for PyTorch’s pre-trained models are as follows：

https://pytorch.org/vision/0.20/models.html

Classification example code:

```
from torchvision.io import decode_image
from torchvision.models import resnet50, ResNet50_Weights

img = decode_image("test/assets/encode_jpeg/grace_hopper_517x606.jpg")

# Step 1: Initialize model with the best available weights
weights = ResNet50_Weights.DEFAULT
model = resnet50(weights=weights)
model.eval()

# Step 2: Initialize the inference transforms
preprocess = weights.transforms()

# Step 3: Apply inference preprocessing transforms
batch = preprocess(img).unsqueeze(0)

# Step 4: Use the model and print the predicted category
prediction = model(batch).squeeze(0).softmax(0)
class_id = prediction.argmax().item()
score = prediction[class_id].item()
category_name = weights.meta["categories"][class_id]
print(f"{category_name}: {100 * score:.1f}%")
```

Object detection example code:

```
from torchvision.io.image import decode_image
from torchvision.models.detection import fasterrcnn_resnet50_fpn_v2,
FasterRCNN_ResNet50_FPN_V2_Weights
from torchvision.utils import draw_bounding_boxes
from torchvision.transforms.functional import to_pil_image

img = decode_image("test/assets/encode_jpeg/grace_hopper_517x606.jpg")

# Step 1: Initialize model with the best available weights
weights = FasterRCNN_ResNet50_FPN_V2_Weights.DEFAULT
model = fasterrcnn_resnet50_fpn_v2(weights=weights, box_score_thresh=0.9)
model.eval()

# Step 2: Initialize the inference transforms
preprocess = weights.transforms()

# Step 3: Apply inference preprocessing transforms
batch = [preprocess(img)]

# Step 4: Use the model and visualize the prediction
prediction = model(batch)[0]
labels = [weights.meta["categories"][i] for i in prediction["labels"]]
box = draw_bounding_boxes(img, boxes=prediction["boxes"],
                          labels=labels,
                          colors="red",
                          width=4, font_size=30)
im = to_pil_image(box.detach())
im.show()
```

Evaluation datasets:

The evaluation datasets for classification and object detection tasks are available at Teams in Group Project folder.

The datasets are subset of ImageNet dataset. In the classification folder, it has 30 images including 20 images of airplanes and 10 other images. In the detection folder, it has 10 images including 55 airplanes.

If you have any concerns regarding the team assignments, please inform Xie Sixiang before 24 March 2026. No adjustments will be allowed after this date.

Notes:
You can develop the project base on PyTorch example code. You have to improve the code for the evaluation of multiple images and save the results.

In the report, you need to at least summarize the paper including background and method and present the quantitative results you obtained. In addition, you need show screen record to show the code is running correctly.

Please read the assigned paper carefully. In the background, you need include all the assigned paper for classification and object detection task. You just need focus on your selected task and model in the rest part.

For the experimental evaluation, in the classification task, you need to count the number and proportion of classified airplane images. In the object detection task, you only need to count the number and proportion of airplanes detected in each image and the total number of images.

2) Submission

Deadline: 23:59, 10th April 2026 (Late submission will not be accepted).
The oral presentation will be held in the last week of the course.

Note: Only one team member needs to upload 3 materials (code, report and slides) via Teams. Please keep the size of all materials as small as possible (less than 100M).

At the beginning of the code, report and presentation, please provide full information of all group members (name and ID) and the contribution of each member.

All documents (except for code) must be converted to PDF files!!! Don't submit word and ppt files!!

Please submit the code (optionally with demo/screen shots/etc..), report (<=4 pages, 12pt, A4, Normal margin), and slides and in-class oral presentation (total time<=15min for each group, including Q&A time, presentation time<= 12min in total, in principle, each team member should equally share the presentation time).

3) Marking Criteria

Code (40 marks)

(1) Executable code without syntax/runtime errors. (10marks)
(2) Well-commented code and clearly explained key parameters. (5 marks)
(3) complete workflow (data loading → model inference → results output) (15 marks)
(4) Evaluate the model (10 marks)

Report (40 marks)

(1) Contains at least three parts: background, methods and experiments and have correct format. (10 marks)
(2) The report has an in-depth analysis or comparison of the principles, advantages, shortcomings of the model. (10 marks)
(3) A complete and detailed introduction to the specified papers. (10 marks)
(4) Obtain the similar quantitative results to ground truth. (10 marks)

Presentation (20 marks)

(1) Presentation content: The presentation is clear, accurately and logically describing the background, methods, and experiments. (5 marks)
(2) Presentation visualization: The slides are clear, with rich and vivid pictures and visualization results. (5 marks)
(3) Presentation Performance: The oral presentation is fluent and academic. (5 marks)
(4) Q&A Performance: Precise answers to technical questions. (5 marks)

Paper List:

The following research papers cover relevant deep learning techniques. All papers have open-sourced code on GitHub.

[1] He K, Zhang X, Ren S, et al. Deep residual learning for image recognition[C]//Proceedings of the IEEE conference on computer vision and pattern recognition. 2016: 770-778. https://openaccess.thecvf.com/content_cvpr_2016/papers/He_Deep_Residual_Learning_CVPR_2016_paper.pdf

[2] Simonyan K, Zisserman A. Very deep convolutional networks for large-scale image recognition[J].ICLR, 2014. https://arxiv.org/pdf/1409.1556

[3] Liu W, Anguelov D, Erhan D, et al. Ssd: Single shot multibox detector[C]//Computer Vision–ECCV 2016: 14th European Conference, Amsterdam, The Netherlands, October 11–14, 2016, Proceedings, Part I 14. Springer International Publishing, 2016: 21-37. https://arxiv.org/pdf/1512.02325

[4] Ren S, He K, Girshick R, et al. Faster R-CNN: Towards real-time object detection with region proposal networks[J]. IEEE transactions on pattern analysis and machine intelligence, 2016, 39(6): 1137-1149. https://ieeexplore.ieee.org/abstract/document/7485869